BLOOD CELL CLASSIFICATION USING DEEP LEARNING

A Project Report

Submitted in partial fulfillment of the requirements
for the award of degree

Bachelor of Technology
in
Computer Science and Engineering

Submitted By:
P. Vikram Simha Reddy

Date: March 2026

ABSTRACT

Accurate identification of white blood cells is essential for diagnosing infections, immune disorders, and various hematological conditions. Traditional microscopic examination is time-consuming and depends heavily on the expertise of medical professionals, which may lead to inconsistencies in results. This project presents an automated Blood Cell Classification System that leverages deep learning techniques to classify microscopic images of white blood cells into categories such as Eosinophils, Lymphocytes, Monocytes, and Neutrophils.

The proposed system utilizes a Convolutional Neural Network (CNN) with a transfer learning approach based on the MobileNetV2 architecture to extract meaningful features from blood cell images. Image preprocessing techniques including resizing and normalization are applied to enhance model performance. The trained model is integrated into a Flask-based web application that allows users to upload blood smear images and receive predictions along with confidence scores.

The system demonstrates reliable classification performance on the testing dataset and provides fast, consistent results compared to manual analysis. By combining deep learning with a user-friendly web interface, the application offers a practical solution for supporting medical diagnostics and laboratory analysis. The project highlights the potential of artificial intelligence in healthcare by improving efficiency, reducing human error, and enabling scalable automated screening systems. Future enhancements may include expanding the dataset, incorporating additional cell types, and deploying the model in real-time clinical environments.

Keywords

Deep Learning, Blood Cell Classification, Convolutional Neural Network (CNN), MobileNetV2, Medical Image Processing, Transfer Learning, Flask Web Application, Healthcare Automation.

1. INTRODUCTION

Blood plays an important role in maintaining human health. The examination of blood cells helps doctors diagnose infections, immune system disorders, anemia, leukemia, and many other medical conditions. White blood cells are especially important because they help the body fight against diseases. These cells are generally classified into different types such as Eosinophils, Lymphocytes, Monocytes, and Neutrophils. Each type has a specific function in the immune system.

Traditionally, blood cell identification is performed manually by laboratory technicians using a microscope. This process requires skill, experience, and time. Manual analysis may sometimes lead to errors due to fatigue or variations in human judgment. Therefore, there is a need for an automated and reliable system that can assist medical professionals in analyzing blood samples more efficiently.

With the rapid development of Artificial Intelligence and Deep Learning, image classification has become highly accurate and efficient. Convolutional Neural Networks (CNNs) are widely used for medical image analysis because they can automatically learn important features from images. In this project, a deep learning model based on the MobileNetV2 architecture is used to classify blood cell images.

The trained model is integrated into a web application developed using Flask. Users can upload a blood cell image and receive the predicted cell type along with a confidence percentage. This system aims to reduce manual effort, improve accuracy, and support healthcare professionals in faster diagnosis.

2. LITERATURE REVIEW

Medical image analysis has gained significant importance with the advancement of Artificial Intelligence and Machine Learning. Many researchers have focused on using deep learning techniques for automatic classification of blood cells to improve diagnostic accuracy and reduce manual workload. Convolutional Neural Networks (CNNs) are widely used in image classification tasks because they can automatically extract important features from images without the need for manual feature engineering.

Earlier methods for blood cell classification relied on traditional image processing techniques such as edge detection, thresholding, and feature extraction based on shape, size, and color. Although these methods provided moderate accuracy, they required careful feature selection and were sensitive to variations in lighting and image quality.

Recent studies show that deep learning models, especially CNN-based architectures, perform better than traditional methods. Pre-trained models such as VGG16, ResNet, and MobileNet have been successfully applied in medical image classification through transfer learning. Transfer learning allows models trained on large datasets to be adapted for specific tasks with smaller datasets, improving performance and reducing training time.

MobileNetV2 is particularly useful because it provides high accuracy with lower computational cost. This makes it suitable for web-based and real-time applications. Several research works have demonstrated that automated blood cell classification systems can assist laboratory technicians by providing fast and consistent results.

Based on these studies, this project adopts a CNN model using MobileNetV2 architecture to achieve efficient and accurate classification of white blood cells through a user-friendly web interface.

.

3. SYSTEM ARCHITECTURE

The Blood Cell Classification System follows a structured and modular architecture to ensure efficient processing, accurate prediction, and smooth user interaction. The system is designed using Deep Learning for image classification and Flask for web deployment. The architecture consists of interconnected modules that work together to perform classification from image input to result output.

1. Input Layer – Image Acquisition

The system begins with image input from the user. The user uploads a microscopic blood smear image through the web interface. The image is received by the Flask server and temporarily stored for processing. Only image file formats such as JPG or PNG are accepted to maintain compatibility.

2. Image Preprocessing Layer

Once the image is uploaded, preprocessing is performed to prepare it for prediction. This includes:

- Resizing the image to match the input size required by MobileNetV2.
- Converting the image into array format.
- Applying normalization and preprocessing functions.
- Expanding image dimensions to match model input shape.

This step ensures that the uploaded image is consistent with the format used during model training.

3. Feature Extraction Layer

The core of the system uses MobileNetV2 as a feature extractor. MobileNetV2 is a lightweight Convolutional Neural Network that extracts important image features such as edges, shapes, and patterns automatically.

Instead of manually selecting features, the CNN identifies relevant characteristics of each blood cell type during training. This improves accuracy and reduces human effort.

4. Classification Layer

After feature extraction, the final classification layer predicts the blood cell category. The system classifies images into:

- Eosinophils
- Lymphocytes
- Monocytes
- Neutrophils

The model outputs probability values for each class. The highest probability is selected as the final prediction. The confidence percentage is also calculated and displayed.

5. Model Storage and Loading Module

After training, the model is saved as a file. During runtime, the Flask application loads this trained model. This avoids retraining every time the application runs and ensures faster predictions.

6. Web Application Module

The Flask framework manages user interaction and system communication. It includes:

- Home Page (Image Upload Interface)
- Result Page (Prediction Output Display)

When the user uploads an image:

1. The image is sent to the backend.
2. Preprocessing is applied.
3. The model predicts the class.
4. The result and confidence percentage are displayed.

7. Output Layer

The final output includes:

- Predicted Blood Cell Type
- Confidence Percentage

The results are displayed clearly on the result page, making the system user-friendly and easy to understand.

4. METHODOLOGY

The methodology of the Blood Cell Classification System explains the step-by-step process followed to design, train, and deploy the deep learning model. The complete process includes dataset preparation, preprocessing, model training, evaluation, and integration into a web application.

1. Dataset Collection

The project uses a labeled dataset containing microscopic images of white blood cells. The dataset is organized into separate folders based on cell types such as Eosinophils, Lymphocytes, Monocytes, and Neutrophils. The images are divided into training and testing sets to ensure proper evaluation of the model.

2. Data Preprocessing

Before training the model, preprocessing is applied to improve consistency and accuracy. The following steps are performed:

- Images are resized to match the input size required by MobileNetV2.
- Images are converted into numerical array format.
- Pixel values are normalized using MobileNetV2 preprocessing functions.
- Data is structured in batches for efficient training.

Preprocessing ensures that all images are uniform and suitable for model learning.

3. Model Selection

The project uses MobileNetV2, a pre-trained Convolutional Neural Network model. Transfer learning is applied to utilize previously learned features from large image datasets.

Instead of training the model from scratch, the base layers of MobileNetV2 are used for feature extraction, and custom classification layers are added for blood cell prediction.

4. Model Training

The training process involves feeding preprocessed images into the model. During training:

- The model learns important features such as shape, texture, and color differences.

- Loss and accuracy are monitored.
- The optimizer adjusts weights to reduce classification error.

The model is trained for multiple epochs until satisfactory performance is achieved.

5. Model Evaluation

After training, the model is tested using the testing dataset. Performance metrics such as accuracy are calculated. This helps in determining how well the model generalizes to new images.

6. Model Saving

Once trained, the model is saved as a file. This allows it to be reused for predictions without retraining every time.

7. Web Application Integration

The trained model is integrated into a Flask-based web application. The methodology for prediction includes:

1. User uploads an image through the home page.
2. The image undergoes preprocessing.
3. The model predicts the blood cell type.
4. The confidence percentage is calculated.
5. The result is displayed on the result page.

---

8. Result Generation

The final output shows the predicted class along with the probability score. This ensures transparency and reliability in classification.

5. TOOLS AND TECHNOLOGIES USED

The Blood Cell Classification System is developed using modern programming tools, deep learning frameworks, and web technologies. The following tools and technologies were used during the development and deployment of the project:

1. Programming Language

Python
Python is used as the primary programming language for implementing the deep learning model and the web application. It provides extensive libraries and strong support for machine learning and image processing.

2. Deep Learning Framework

TensorFlow and Keras
TensorFlow is used as the backend deep learning framework, while Keras provides a high-level API for building and training the Convolutional Neural Network. These frameworks are used to:

- Build the MobileNetV2-based model
- Train and evaluate the model
- Save and load the trained model

3. Model Architecture

MobileNetV2
MobileNetV2 is a pre-trained Convolutional Neural Network used for transfer learning. It is lightweight, efficient, and suitable for image classification tasks. It helps in extracting meaningful features from blood cell images while reducing computational cost.

4. Web Framework

Flask
Flask is used to develop the web application interface. It connects the trained model with the front-end interface and handles image upload, processing, and result display.

5. Image Processing Libraries

OpenCV (cv2)
OpenCV is used for reading and processing images before feeding them into the model.

NumPy
NumPy is used for numerical operations, image array handling, and data manipulation.

6. Front-End Technologies

HTML and CSS are used to design the Home page and Result page of the web application, providing a simple and user-friendly interface.

7. Development Environment

- VS Code / Any Python IDE
- Web Browser for testing the Flask application

These tools and technologies collectively ensure accurate model training, smooth web integration, and efficient prediction of blood cell types.

6. RESULTS

The Blood Cell Classification System was successfully developed and tested using a labeled dataset of microscopic white blood cell images. The system was trained using the MobileNetV2 architecture with transfer learning, which enabled efficient feature extraction and improved classification performance.

1. Model Performance

After training the model with the training dataset, evaluation was performed using the testing dataset. The model achieved high classification accuracy in identifying different types of white blood cells, including:

- Eosinophils
- Lymphocytes
- Monocytes
- Neutrophils

The use of MobileNetV2 helped in reducing computational complexity while maintaining strong prediction performance. The loss value decreased steadily during training, and accuracy improved with each epoch, indicating effective learning.

2. Prediction Output

The trained model was integrated into a Flask-based web application. The system was tested by uploading different blood cell images through the web interface.

For each uploaded image, the system successfully:

1. Processed the image using preprocessing techniques.
2. Predicted the blood cell type.
3. Displayed the classification result.
4. Showed the confidence percentage of the prediction.

The results were displayed clearly on the result page, making the system easy to use and understand.

3. System Functionality

The application performed consistently without errors during testing. The image upload feature worked correctly, and predictions were generated quickly. The model loading mechanism ensured that predictions were made without retraining the model each time.

4. Overall Outcome

The project demonstrates that deep learning techniques can effectively classify blood cell images with reliable accuracy. The integration of the trained model into a web application makes the system practical and accessible.

The results confirm that the proposed system can assist medical professionals by providing fast, automated, and consistent blood cell classification.

7. ADVANTAGES

The Blood Cell Classification System offers several advantages by combining deep learning techniques with a web-based interface. The main advantages of the proposed system are as follows:

1. Reduced Manual Effort

The system automates the process of identifying white blood cells from microscopic images. This reduces the need for continuous manual observation under a microscope and minimizes human workload.

2. Improved Accuracy

By using a Convolutional Neural Network based on MobileNetV2, the system can accurately extract important image features such as shape, texture, and color. This improves classification accuracy compared to traditional manual methods.

3. Faster Diagnosis

The system provides predictions within seconds after image upload. This saves time in laboratory environments and supports quicker medical decision-making.

4. Consistent Results

Unlike manual analysis, which may vary from person to person, the automated system provides consistent results for similar input images. This improves reliability in classification.

5. Lightweight and Efficient Model

MobileNetV2 is a lightweight model that requires less computational power compared to larger deep learning architectures. This makes the system suitable for web-based applications and real-time use.

6. User-Friendly Interface

The Flask-based web application allows users to easily upload images and view results. The interface is simple and does not require advanced technical knowledge to operate.

7. Scalable and Extendable

The system can be extended to include more blood cell types or integrated into hospital management systems. It can also be deployed on cloud platforms for wider accessibility.

8. Cost-Effective Solution

Once developed and deployed, the system reduces dependency on extensive manual analysis, potentially lowering operational costs in laboratories.

These advantages demonstrate that the proposed system is efficient, reliable, and practical for supporting automated medical image classification.

8. LIMITATIONS

Although the Blood Cell Classification System provides accurate and fast predictions, there are certain limitations associated with the current implementation.

1. Dependence on Dataset Quality

The performance of the model depends heavily on the quality and size of the dataset used for training. If the dataset contains low-resolution or imbalanced images, the model accuracy may decrease.

2. Limited Cell Categories

The current system is trained to classify only four types of white blood cells: Eosinophils, Lymphocytes, Monocytes, and Neutrophils. It cannot identify other blood components such as red blood cells, platelets, or abnormal cells unless the model is retrained with additional data.

3. Requires Proper Image Input

The system performs best when clear and properly captured microscopic images are provided. Blurry or poorly illuminated images may lead to incorrect predictions.

4. Computational Requirements for Training

Although MobileNetV2 is lightweight, training the model still requires sufficient computational resources such as a system with good RAM and processing capability. Training on very large datasets may require GPU support.

5. Not a Complete Diagnostic Tool

The system only classifies blood cell types and does not provide a full medical diagnosis. Final medical decisions must always be made by qualified healthcare professionals.

6. No Real-Time Microscope Integration

The current implementation requires manual image upload through the web application. It is not directly connected to laboratory microscopes for automatic real-time image capture.

These limitations highlight areas where the system can be improved in future versions to enhance performance, usability, and clinical application.

9. FUTURE SCOPE

The Blood Cell Classification System can be further improved and expanded in several ways to enhance its practical application in the medical field.

Firstly, the dataset can be expanded to include more types of blood cells and abnormal cells. This would allow the system to detect a wider range of medical conditions. Including red blood cells, platelets, and cancerous cells would make the system more comprehensive.

Secondly, the model accuracy can be improved by training on a larger and more diverse dataset. Using advanced deep learning architectures or ensemble models may further enhance classification performance.

Thirdly, the system can be integrated directly with digital microscopes for real-time image capture and automatic analysis. This would eliminate the need for manual image upload and make the system more suitable for laboratory environments.

The project can also be deployed on cloud platforms, allowing hospitals and diagnostic centers to access the system remotely. A mobile application version could be developed to increase accessibility.

In addition, performance metrics such as confusion matrix, precision, recall, and F1-score can be incorporated for detailed analysis. Security features can also be added to ensure patient data privacy.

With continuous improvements, this system has strong potential to become a supportive tool in automated medical diagnostics.

10. CONCLUSION

The Blood Cell Classification System successfully demonstrates the application of deep learning techniques in medical image analysis. By using a Convolutional Neural Network based on MobileNetV2 and integrating it with a Flask web application, the system provides accurate and efficient classification of white blood cell images.

The project reduces manual effort, improves consistency, and delivers fast prediction results with confidence percentage. The structured dataset, preprocessing techniques, and transfer learning approach contribute to reliable model performance.

Although the system has certain limitations, it provides a strong foundation for automated healthcare solutions. With future enhancements such as real-time integration and expanded datasets, the system can play an important role in supporting laboratory diagnostics.

Overall, this project highlights the importance of Artificial Intelligence in healthcare and demonstrates how deep learning can be used to build practical, user-friendly, and scalable medical applications.

REFERENCES

1. Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv preprint arXiv:1704.04861.
2. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
4. TensorFlow Developers. (2023). *TensorFlow Documentation*. Available at: https://www.tensorflow.org/
5. Keras Documentation. (2023). *Keras: Deep Learning for Humans*. Available at: https://keras.io/
6. Flask Documentation. (2023). *Flask Web Framework Documentation*. Available at: https://flask.palletsprojects.com/
7. OpenCV Documentation. (2023). *Open Source Computer Vision Library*. Available at: https://opencv.org/
8. NumPy Developers. (2023). *NumPy Documentation*. Available at: https://numpy.org/

Github Link :

https://github.com/ Vikram-tech512/Blood-Cell-Detection

Vikram-tech512/Blood-Cell-Detection